

TASKER

Roteiro completo de desenvolvimento

Um app de tarefas premium no nível de Todoist e Things 3 — começando como webapp (PWA) para celular e desktop, com caminho claro para virar app nativo no futuro.

Stack: Vite + React + TypeScript • Tailwind • Supabase • Netlify • PWA
Junho de 2026

1. Visão geral do projeto

O TASKER combina o melhor das suas duas referências. Do **Todoist**, você quer a velocidade: entrada rápida de tarefas em linguagem natural ("pagar conta amanhã 9h"), organização por projetos e etiquetas, e a sensação de que nada escapa. Do **Things 3**, você quer a calma: tipografia impecável, espaçamento generoso, animações suaves, e a estrutura Hoje / Em breve / A qualquer momento que faz o app parecer um lugar agradável de estar, não uma lista de cobranças.

A estratégia é construir um **PWA (Progressive Web App)**: um webapp que roda no navegador do desktop e pode ser "instalado" na tela inicial do celular, com ícone próprio, tela cheia e funcionamento offline. Isso permite lançar rápido com uma única base de código. Quando quiser virar app de verdade nas lojas, o mesmo código entra no **Capacitor** e vira app iOS/Android com pouquíssimo retrabalho.

O que define "premium" neste projeto

- **Velocidade percebida:** tudo responde em menos de 100ms. Atualizações otimistas (a interface muda antes do servidor confirmar).
- **Microinterações:** checkbox com animação de conclusão satisfatória, listas que reordenam suavemente, transições entre telas.
- **Tipografia e espaçamento:** uma fonte de qualidade (Inter ou similar), hierarquia clara, muito respiro em volta dos elementos.
- **Atalhos de teclado no desktop:** Q para adicionar tarefa, setas para navegar, Enter para editar, Cmd/Ctrl+K para busca.
- **Estados vazios bonitos:** quando não há tarefas, a tela celebra em vez de parecer quebrada.
- **Modo escuro de verdade:** pensado desde o início, não invertido no final.

Regra de ouro do processo

Construa em **fases pequenas e verificáveis**. Em cada fase, peça uma coisa, teste no navegador, só então avance. Nunca peça "faça o app inteiro" — o resultado fica genérico e cheio de bugs. O roteiro abaixo já divide tudo em pedidos do tamanho certo, com o prompt pronto para cada um.

2. Stack técnica (e por quê)

Camada	Escolha	Por quê
Build / framework	Vite + React + TypeScript	Rápido, simples de hospedar no Netlify como site estático, ecossistema gigante. TypeScript evita bugs bobos.
Estilo	Tailwind CSS	Permite construir um design system consistente com tokens (cores, espaçamentos) sem CSS espalhado.
Animações	Framer Motion	É o que dá a sensação Things 3: transições de layout, reordenação animada, gestos.
Estado local	Zustand	Leve e direto. Guarda as tarefas em memória e sincroniza com o banco por trás.
Backend	Supabase	Você já tem conta. Resolve login, banco Postgres, sincronização em tempo real e regras de segurança (RLS) sem você escrever servidor.
Offline / instalável	vite-plugin-pwa	Transforma o site em PWA: ícone na tela inicial, splash screen, cache offline.
Deploy	Netlify	Você já tem conta. Conecta no GitHub e publica sozinho a cada commit.
Futuro nativo	Capacitor	Empacota exatamente este código como app iOS/Android quando chegar a hora.

Onde desenvolver: Claude Code é o ideal

Um projeto deste tamanho não cabe bem em conversas de chat com artifacts — você precisa de um projeto real com dezenas de arquivos, Git, e iteração rápida. O caminho recomendado é o **Claude Code** (no terminal, no VS Code ou no app desktop do Claude): ele cria os arquivos direto no seu repositório, roda o projeto, vê erros e corrige. Os prompts deste roteiro foram escritos para serem usados lá, mas funcionam em qualquer lugar onde o Claude possa criar arquivos.

Skills que ajudam no processo

- **frontend-design**: a skill mais importante para este projeto. Garante interface com cara própria e intencional, em vez de visual genérico de template. No Claude Code, peça explicitamente: "use a skill frontend-design".
- **product-self-knowledge**: útil se você for usar a API da Anthropic dentro do app (por exemplo, um assistente que organiza suas tarefas).
- No `claude.ai`, as skills de documentos (pdf, docx) servem para gerar materiais como este roteiro — não são necessárias para o código do app em si.

3. As fases do projeto

São 8 fases. Cada uma tem objetivo, critério de "pronto" e um prompt completo para copiar e colar. Termine e teste uma fase antes de começar a próxima. Faça um commit no Git ao final de cada fase — se algo der errado depois, você volta para um ponto seguro.

FASE 0 — Fundação do projeto

Objetivo: Criar o repositório, o projeto Vite com TypeScript, Tailwind e a estrutura de pastas. Ao final, um "Hello TASKER" rodando localmente e publicado no Netlify.

PROMPT — Fase 0

Crie um novo projeto chamado TASKER: um app de tarefas premium inspirado em Todoist e Things 3, que será um PWA (webapp instalável) para desktop e celular.

Nesta primeira etapa, faça apenas a fundação:

1. Projeto Vite + React + TypeScript.
2. Tailwind CSS configurado.
3. Instale: zustand, framer-motion, @supabase/supabase-js, date-fns, lucide-react.
4. Estrutura de pastas: src/components, src/features/tasks, src/features/projects, src/stores, src/lib, src/styles.
5. Uma página inicial simples escrita "TASKER" centralizada, só para validar que tudo roda.
6. README com instruções de como rodar (npm run dev).

Não crie nenhuma funcionalidade ainda. Quero validar a base primeiro.

Depois do prompt: rode `npm run dev`, confirme que abre no navegador. Crie o repositório no GitHub, faça o primeiro commit e conecte o repositório ao Netlify (build command: `npm run build`, publish directory: `dist`). A partir daqui, todo push publica automaticamente.

Pronto quando: o site abre no endereço do Netlify, no celular e no desktop.

FASE 1 — Design system

Objetivo: Definir a identidade visual do TASKER antes de qualquer funcionalidade. Isso é o que separa um app premium de um template.

PROMPT — Fase 1

Use a skill frontend-design. Vamos criar o design system do TASKER antes das funcionalidades. Referências de qualidade: a calma e a tipografia do Things 3, a clareza do Todoist. Mas o TASKER deve ter identidade própria, não ser uma cópia.

Crie:

1. Tokens no Tailwind: paleta com um neutro quente para fundos, um azul-tinta como cor de acento, cores semânticas (vencido, hoje, concluído). Defina também a versão dark de cada token – o modo escuro nasce junto, não depois.
2. Tipografia: fonte Inter (variável), escala de tamanhos definida, títulos com peso forte e tracking levemente negativo.
3. Componentes base em src/components/ui: Button, Input, Checkbox (com animação de conclusão usando framer-motion – esse checkbox é a alma do app, capriche), Modal, Tooltip, Tag.
4. Uma página /design-system que mostra todos os componentes nos dois temas, para eu revisar.

Critérios: cantos arredondados consistentes, sombras sutis em camadas, foco visível para teclado, áreas de toque de no mínimo 44px pensando no celular.

Pronto quando: você olha a página /design-system e sente que "parece um app caro". Se não sentir, itere aqui — peça ajustes de cor, espaçamento e animação ANTES de seguir. É a fase mais barata para mudar o visual.

FASE 2 — Núcleo de tarefas (local, sem servidor)

Objetivo: O coração do app funcionando 100% no navegador, salvando localmente. Sem login ainda — isso elimina metade da complexidade enquanto o produto toma forma.

PROMPT — Fase 2

Agora o núcleo do TASKER. Tudo local por enquanto (Zustand + persistência em IndexedDB via idb-keyval). Sem backend nesta fase.

Modelo de dados:

- Task: id, título, notas, concluída, data de vencimento (opcional), projeto (opcional), etiquetas, prioridade (p1-p4), ordem, criadaEm, concluídaEm.
- Project: id, nome, cor, ordem.

Telas (com sidebar no desktop e navegação inferior no celular):

1. Caixa de entrada: tarefas sem projeto.
2. Hoje: vencendo hoje + atrasadas (atrasadas em seção própria, em vermelho discreto).
3. Em breve: agrupadas por dia nos próximos 7 dias, depois por mês.
4. Uma tela por projeto.

Funcionalidades:

- Adicionar tarefa rápida: botão + e tecla Q abrem um campo no topo da lista.
- Clicar na tarefa expande edição inline (estilo Things 3): notas, data, projeto, etiquetas, prioridade.
- Concluir com a animação do checkbox da Fase 1; a tarefa some da lista com animação suave (framer-motion layout).
- Arrastar para reordenar.
- Deletar com undo (toast "Tarefa excluída – Desfazer" por 5 segundos).

Tudo deve responder instantaneamente. Capriche nos estados vazios de cada tela.

Pronto quando: você consegue usar o TASKER de verdade no dia a dia, mesmo que só neste navegador. Use por 2-3 dias antes de avançar — os incômodos que você sentir viram os pedidos de ajuste mais valiosos do projeto.

FASE 3 — Interações premium

Objetivo: Transformar o app funcional em app gostoso de usar. É aqui que ele alcança as referências.

PROMPT — Fase 3

Vamos elevar as interações do TASKER ao nível Todoist/Things 3:

1. Entrada em linguagem natural no campo de adicionar tarefa: "pagar internet amanhã", "reunião segunda 14h #trabalho p1" – interprete data, hora, projeto (#) e prioridade (p1-p4) enquanto digito, destacando os trechos reconhecidos com chips coloridos no próprio campo. Use date-fns, em português.
2. Busca global com Cmd/Ctrl+K: paleta de comandos que busca tarefas e projetos e também executa ações ("ir para Hoje", "novo projeto").
3. Atalhos de teclado no desktop: Q (nova tarefa), setas para navegar na lista, Enter (editar), E (concluir), 1-4 (prioridade), T (hoje), digite ? para ver a lista de atalhos.
4. No celular: deslizar tarefa para a direita conclui, para a esquerda agenda. Toque longo entra em modo seleção múltipla.
5. Som sutil e opcional ao concluir tarefa (desligado por padrão, com toggle nas configurações).

Performance: nada de animação pode travar; use transform/opacity, nunca anime layout via top/left.

Pronto quando: você mostra o app para alguém e a pessoa pergunta "que app é esse?".

FASE 4 — Supabase: conta e sincronização

Objetivo: Login e sincronização entre dispositivos. As tarefas locais migram para a nuvem no primeiro login.

PROMPT — Fase 4 (parte A: banco)

Vamos conectar o TASKER ao Supabase. Primeiro o banco.

Escreva a migração SQL para eu rodar no SQL Editor do Supabase:

- Tabela projects: id uuid, user_id (referencia auth.users), name, color, sort_order, created_at.
- Tabela tasks: id uuid, user_id, project_id (nullable, FK), title, notes, completed boolean, completed_at, due_date date (nullable), due_time time (nullable), priority smallint, labels text[], sort_order, created_at, updated_at.
- Índices em (user_id, due_date) e (user_id, project_id).
- RLS habilitado nas duas tabelas, com políticas para o usuário só ver/criar/editar/apagar as próprias linhas. Isso é obrigatório.

Explique passo a passo onde clico no painel do Supabase para rodar isso e onde pego a URL e a anon key do projeto.

PROMPT — Fase 4 (parte B: app)

Banco criado. Agora integre o app:

1. Login por e-mail com link mágico e login com Google, numa tela de boas-vindas bonita que mostra o valor do app.
2. As chaves do Supabase vão em variáveis de ambiente (.env local e variáveis no painel do Netlify), nunca no código.
3. Sincronização com atualização otimista: toda ação atualiza a interface na hora e envia ao Supabase em segundo plano; em caso de erro, reverte e mostra toast. A interface NUNCA espera o servidor.
4. Realtime do Supabase: mudanças feitas em outro dispositivo aparecem nesta tela em segundos.
5. Migração: no primeiro login, ofereça importar as tarefas locais (IndexedDB) para a conta.
6. É possível continuar usando sem conta (modo local), com um aviso gentil de que sem login não há sincronização.

Pronto quando: você cria uma tarefa no celular e ela aparece no desktop em segundos — e as políticas RLS estão ativas (teste: um segundo usuário não vê suas tarefas).

FASE 5 — PWA: instalável e offline

Objetivo: Transformar o site em app instalável com funcionamento offline.

PROMPT — Fase 5

Transforme o TASKER em PWA completo usando vite-plugin-pwa:

1. Manifest: nome TASKER, ícones em todos os tamanhos (gere a partir de um ícone base – crie um ícone simples e elegante com a inicial T), theme color seguindo o tema claro/escuro, display standalone.
 2. Service worker: o app abre e funciona offline; ações feitas offline entram numa fila e sincronizam quando a conexão volta.
 3. Banner de instalação discreto (no Android/desktop via beforeinstallprompt; no iOS, uma dica de "Adicionar à Tela de Início" mostrada uma única vez).
 4. Aviso elegante quando há nova versão do app, com botão "Atualizar".
 5. No iOS instalado: respeitar as safe areas (notch) e impedir zoom/bounce indesejados.
- Me diga como testar a instalação no Android, no iPhone e no desktop.

Pronto quando: o TASKER está na tela inicial do seu celular, abre em tela cheia, e funciona no modo avião.

FASE 6 — Polimento e qualidade

Objetivo: A camada final que consolida a sensação premium.

PROMPT — Fase 6

Rodada de polimento do TASKER. Revise o app inteiro com olhar crítico:

1. Acessibilidade: navegação completa por teclado, roles ARIA nas listas, contraste AA nos dois temas, respeitar prefers-reduced-motion.
 2. Performance: meça o bundle, faça code-splitting por rota, lazy-load do que não é crítico. Meta: carregar em menos de 2s em 4G; interações abaixo de 100ms.
 3. Telas que faltam: configurações (tema, som, primeiro dia da semana), perfil/conta, tela 404.
 4. Revisão de microcópia em português: mensagens curtas, claras e humanas. Estados vazios com personalidade.
 5. Pente fino visual: alinhamentos, espaçamentos inconsistentes, qualquer coisa "quase certa". Liste o que encontrar e corrija.
- Depois me entregue um checklist do que foi verificado.

FASE 7 — Lançamento

Objetivo: Domínio, verificações finais e publicação oficial.

- Compre um domínio (ex.: tasker.app.br) e conecte no Netlify (Domain settings) — HTTPS é automático.
- No Supabase, adicione o domínio final nas URLs de redirecionamento da autenticação.
- Teste o fluxo completo num dispositivo "limpo": instalar, criar conta, usar offline, sincronizar.
- Peça para 2-3 pessoas usarem por uma semana e anote tudo o que confundir elas.

FASE FUTURA — App nativo com Capacitor

Objetivo: Quando o PWA estiver maduro e você quiser estar nas lojas.

O Capacitor empacota exatamente o seu código web como app iOS e Android. O que muda: notificações push de verdade (lembretes de tarefas), widgets, ícone com badge de contagem. O prompt, quando chegar a hora, será: *"Adicione Capacitor ao TASKER, configure os projetos iOS e Android, e implemente notificações locais para lembretes de tarefas com data e hora."* Como o app já foi construído mobile-first, essa transição é pequena.

4. Como pedir bem (vale para todas as fases)

Princípios dos bons prompts neste projeto

- **Uma fase por vez.** Pedidos grandes geram código genérico. Pedidos focados geram qualidade.
- **Dê referências de sensação, não de cópia:** "a calma do Things 3", "a velocidade do Todoist" — e deixe claro que o TASKER tem identidade própria.
- **Defina critérios verificáveis:** "responde em menos de 100ms", "funciona no modo avião", "toque mínimo de 44px".
- **Quando algo sair errado, descreva o sintoma exato:** "ao concluir uma tarefa no celular, a lista pula para o topo" funciona muito melhor que "está bugado".
- **Peça explicação quando quiser aprender:** "explique o que é RLS e por que essa política protege meus dados".

Prompts de manutenção que você vai usar sempre

PROMPT — Ajuste visual

Olhe a tela [Hoje] com olhar de designer sênior. Compare com o padrão de qualidade de Todoist/Things 3 e liste tudo que está abaixo desse nível (espaçamento, hierarquia, alinhamento, animação). Depois corrija os 5 problemas mais importantes.

PROMPT — Correção de bug

Bug: [descreva o que você fez, o que esperava, e o que aconteceu — se possível com o erro do console].

Antes de corrigir, explique a causa. Depois corrija sem mudar comportamento de outras partes.

PROMPT — Nova funcionalidade

Quero adicionar [funcionalidade] ao TASKER. Antes de implementar, me proponha como ela deve funcionar e onde deve aparecer na interface, mantendo a simplicidade do app. Só implemente depois que eu aprovar a proposta.

5. Checklist de qualidade premium

Antes de considerar qualquer fase concluída

- Funciona bem no celular E no desktop (teste nos dois sempre).
- Nenhuma ação visível demora — a interface responde na hora, o servidor que espere.
- Modo claro e escuro impecáveis.
- Dá para operar só com teclado no desktop.
- Estados vazios, de erro e de carregamento foram desenhados, não esquecidos.
- Textos em português natural, sem jargão técnico vazando para o usuário.
- Commit feito no Git com mensagem clara.

Erros comuns a evitar

- **Pular a Fase 1 (design system)** para "ver funcionalidade logo". É a causa nº 1 de apps com cara genérica.
- **Adicionar Supabase cedo demais.** Backend antes do produto estar definido dobra o custo de cada mudança.
- **Esquecer o RLS no Supabase.** Sem as políticas, qualquer usuário lê os dados de todos. Inegociável.
- **Colocar chaves/API keys no código.** Sempre variáveis de ambiente (.env + painel do Netlify).
- **Acumular muitas mudanças sem commit.** Commits pequenos = poder de voltar atrás.
- **Inchar o escopo:** colaboração, IA, gamificação... anote num arquivo IDEIAS.md e mantenha o foco. Todoist e Things 3 são ótimos pelo que deixaram de fora.

6. Cronograma sugerido

Semana	Entrega
1	Fases 0 e 1 — fundação publicada no Netlify + design system aprovado
2-3	Fase 2 — núcleo de tarefas; você começa a usar o app de verdade
4	Fase 3 — interações premium (linguagem natural, atalhos, gestos)
5	Fase 4 — Supabase: conta e sincronização entre dispositivos
6	Fases 5 e 6 — PWA instalável/offline + rodada de polimento
7	Fase 7 — domínio, testes com pessoas reais, lançamento

O ritmo é seu — o que importa é a ordem das fases e o hábito de testar e commitar a cada etapa. Primeiro passo de hoje: criar o repositório **tasker** no GitHub e rodar o prompt da Fase 0. Bom projeto!